

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	K Deepika	Reg. No.:	192371042
Section / Batch:	CSE 4th year	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	A transportation system calculates distances between bus stops to identify the closest stops for route optimization. The number of stops increases daily. a) Explain how brute-force closest pair works here. b) Derive its time complexity. c) Discuss scalability issues.	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I _____K.Deepika(192371042)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __k.deepika_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding ; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Source code:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <limits>
#include <optional>
#include <algorithm>
#include <cstdint>
```

```
namespace Spatial::Analytics {
```

```
    struct BusStop {
        std::uint64_t id;
        double latitude; // Y-coordinate (deg)
        double longitude; // X-coordinate (deg)
    };
```

```
    struct ProximityResult {
        BusStop stop_a;
        BusStop stop_b;
        double distance_meters;
    };
```

```
    enum class DistanceMetric {
        Euclidean2D,
        HaversineMeters
    };
```

```
    class ClosestPairSolver {
    public:
        ClosestPairSolver() = delete;
```

```

[[nodiscard]] static std::optional<ProximityResult> ComputeClosestPair(
    const std::vector<BusStop>& stops,
    DistanceMetric metric = DistanceMetric::HaversineMeters)
{
    const std::size_t count = stops.size();
    if (count < 2) {
        return std::nullopt;
    }

    double min_metric_value = std::numeric_limits<double>::infinity();
    std::pair<std::size_t, std::size_t> best_pair_indices{0, 0};

    for (std::size_t i = 0; i < count - 1; ++i) {
        for (std::size_t j = i + 1; j < count; ++j) {
            const double dist_val = EvaluateDistance(stops[i], stops[j], metric);

            if (dist_val < min_metric_value) {
                min_metric_value = dist_val;
                best_pair_indices = {i, j};
            }
        }
    }

    double final_distance = min_metric_value;
    if (metric == DistanceMetric::Euclidean2D) {
        final_distance = std::sqrt(min_metric_value);
    }

    return ProximityResult{
        .stop_a = stops[best_pair_indices.first],
        .stop_b = stops[best_pair_indices.second],
        .distance_meters = final_distance
    };
}

```

private:

```

static constexpr double EARTH_RADIUS_METERS = 6371000.0;
static constexpr double DEG_TO_RAD = 3.14159265358979323846 / 180.0;

```

```

[[nodiscard]] inline static double EvaluateDistance(

```

```

    const BusStop& a,
    const BusStop& b,
    DistanceMetric metric) noexcept
{
    if (metric == DistanceMetric::Euclidean2D) {
        const double dx = a.longitude - b.longitude;
        const double dy = a.latitude - b.latitude;
        return (dx * dx) + (dy * dy);
    }

    const double lat1 = a.latitude * DEG_TO_RAD;
    const double lat2 = b.latitude * DEG_TO_RAD;
    const double dlat = (b.latitude - a.latitude) * DEG_TO_RAD;
    const double dlon = (b.longitude - a.longitude) * DEG_TO_RAD;

    const double sin_dlat = std::sin(dlat / 2.0);
    const double sin_dlon = std::sin(dlon / 2.0);

    double h = (sin_dlat * sin_dlat) +
        std::cos(lat1) * std::cos(lat2) * (sin_dlon * sin_dlon);

    // Clamp h to range [0.0, 1.0] to prevent NaN errors from domain violation
    h = std::clamp(h, 0.0, 1.0);

    return 2.0 * EARTH_RADIUS_METERS * std::asin(std::sqrt(h));
}
};
}

int main() {
    using namespace Spatial::Analytics;

    const std::vector<BusStop> fleet_stops = {
        {.id = 1001, .latitude = 40.712776, .longitude = -74.005974}, // Stop A
        {.id = 1002, .latitude = 40.713000, .longitude = -74.006100}, // Stop B (Closest to A)
        {.id = 1003, .latitude = 40.758896, .longitude = -73.985130}, // Stop C
        {.id = 1004, .latitude = 40.782865, .longitude = -73.965355} // Stop D
    };

    const auto result = ClosestPairSolver::ComputeClosestPair(

```

```

        fleet_stops,
        DistanceMetric::HaversineMeters
    );

    if (result.has_value()) {
        std::cout << "--- Proximity Analysis Output ---\n";
        std::cout << "Closest Stop A ID : " << result->stop_a.id << "\n";
        std::cout << "Closest Stop B ID : " << result->stop_b.id << "\n";
        std::cout << "Calculated Distance: " << result->distance_meters << " meters\n";
    } else {
        std::cout << "Error: Less than 2 stops provided.\n";
    }

    return 0;
}

```

OUTPUT:

main.cpp	Run	Output
<pre> 60 return ProximityResult{ 61 .stop_a = stops[best_pair_indices.first], 62 .stop_b = stops[best_pair_indices.second], 63 .distance_meters = final_distance 64 }; 65 } 66 67 private: 68 static constexpr double EARTH_RADIUS_METERS = 6371000.0; 69 static constexpr double DEG_TO_RAD = 3.14159265358979323846 / 180.0; 70 71 [[nodiscard]] inline static double EvaluateDistance(72 const BusStop& a, 73 const BusStop& b, 74 DistanceMetric metric) noexcept 75 { 76 if (metric == DistanceMetric::Euclidean2D) { 77 const double dx = a.longitude - b.longitude; 78 const double dy = a.latitude - b.latitude; 79 return (dx * dx) + (dy * dy); 80 } </pre>	Run	<pre> --- Proximity Analysis Output --- Closest Stop A ID : 1001 Closest Stop B ID : 1002 Calculated Distance: 27.0772 meters === Code Execution Successful === </pre>